



mnist project

xiyun hu, trevor tan, ling casenas, zachary song

pre stuff

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import numpy as np
import matplotlib.pyplot as plt
```

```
from google.colab import drive
drive.mount('/content/gdrive')
```

```
#The line below gets the dataset and downloads it into the specified path
transform = torchvision.transforms.Compose([torchvision.transforms.ToTensor()])
dataset = torchvision.datasets.MNIST('PATH_FOR_TRAINING_SET', download=True, train=True, transform=transform) # con

#PyTorch uses dataloaders which you can learn about here: https://pytorch.org/tutorials/beginner/basics/data\_tutorial.html
#This is a convenient module for training, it automatically prepares your data for training with a given batch size
#The shuffle argument will automatically randomize the order of your data (but will keep inputs with associated out
trainloader = torch.utils.data.DataLoader(dataset, batch_size = 64, shuffle = True)

testdataset = torchvision.datasets.MNIST('PATH_FOR_TRAINING_SET', download=True, train=False, transform=transform)
testloader = torch.utils.data.DataLoader(testdataset, batch_size = 64, shuffle = False)
```

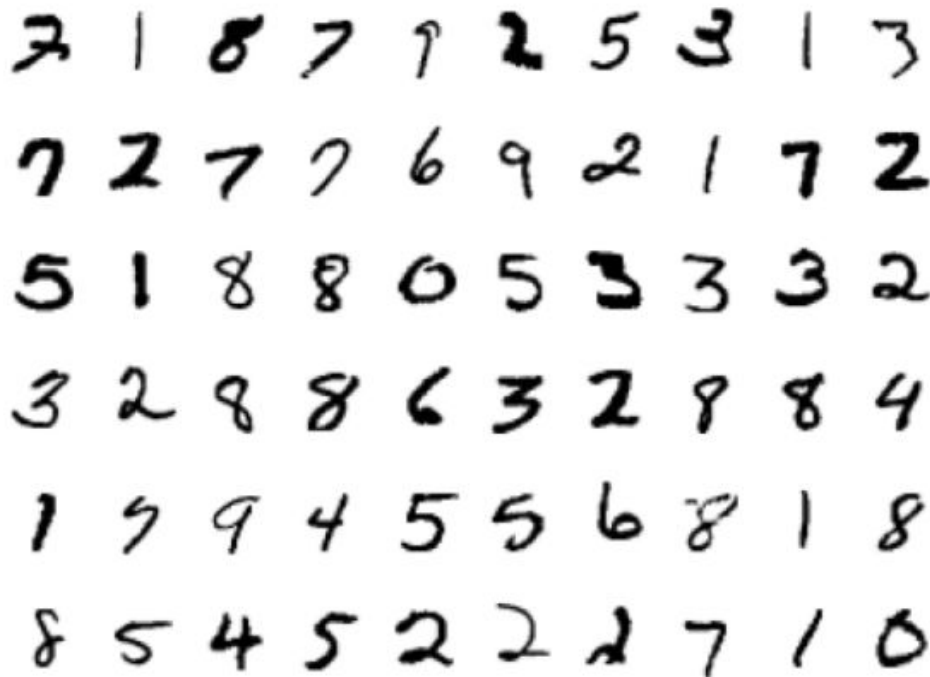
what our data look like

```
dataiter = iter(trainloader)
images, labels = next(dataiter)

print(images.shape)
print(labels.shape)

figure = plt.figure()
num_of_images = 60
for index in range(1, num_of_images + 1):
    plt.subplot(6, 10, index)
    plt.axis('off')
    plt.imshow(images[index].numpy().squeeze(),
```

```
torch.Size([64, 1, 28, 28])
torch.Size([64])
```



unranked architecture

```
class Digit_Classifier(nn.Module):  
    def __init__(self):  
        #Handle some under-the-hood PyTorch stuff  
        super(Digit_Classifier, self).__init__()  
        #Now put your layers below in addition to any of  
        self.fc1 = nn.Linear(28 * 28, 128)  
        self.fc2 = nn.Linear(128, 64)  
        self.fc3 = nn.Linear(64, 10)
```

```
#Think about the following:  
#conv, pool, BatchNorm, Linear, Dropout  
#===== YOUR CODE HERE =====  
self.relu = nn.ReLU()  
self.dropout = torch.nn.Dropout(p = 0.3)
```

```
def forward(self, x):
```

```
    #Now here you add your forward pass, e.g. how  
    #Tips:  
    # 1. Don't forget the ReLU layer when needed  
    # 2. Consider normalization  
    # 3. If you are getting errors look at your  
    # 4. CNN layers take in rectangular (or square)
```

```
    #===== YOUR CODE HERE =====
```

```
    x = torch.flatten(x, start_dim=1)  
    x = self.relu(self.fc1(x))  
    x = self.dropout(x)  
    x = self.relu(self.fc2(x))  
    x = self.dropout(x)  
    x = self.fc3(x)  
    return x;
```

!problem!



ranked architecture

```
class Digit_Classifier(nn.Module):
    def __init__(self):
        #Handle some under-the-hood PyTorch stuff
        super(Digit_Classifier, self).__init__()
        #Now put your layers below in addition to any c
        self.fc1 = nn.Linear(28 * 28, 128)
        self.fc2 = nn.Linear(128, 128)
        self.fc3 = nn.Linear(128, 64)
        self.fc4 = nn.Linear(64, 10)

        #Think about the following:
        #conv, pool, BatchNorm, Linear, Dropout
        #===== YOUR CODE HERE =====
        self.relu = nn.ReLU()
        self.dropout = torch.nn.Dropout(p = 0.3)
        self.proj_784_to_128 = nn.Linear(28*28, 128)
        #===== END =====
```

```
x_flat = torch.flatten(x, start_dim=1)
h1 = self.relu(self.fc1(x_flat))
h1 = self.dropout(h1)
h1 = h1 + self.proj_784_to_128(x_flat)

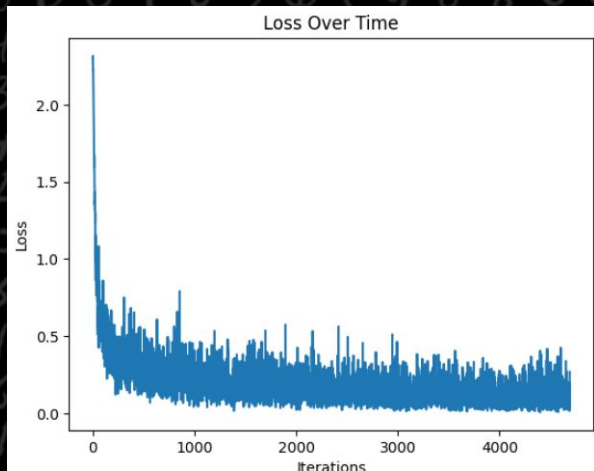
h2 = self.relu(self.fc2(h1))
h2 = self.dropout(h2)
h2 = h2 + h1

h3 = self.relu(self.fc3(h2))
h3 = self.dropout(h3)

out = self.fc4(h3)
return out
```

training

in the future: maybe
grid search?



```
for i in range(num_epochs):
    epoch_loss = 0.0
    for batchIndex, (images, labels) in enumerate(trainloader):
        images, labels = images.to(device), labels.to(device)

        #===== YOUR CODE HERE =====

        # do any preprocessing, for example flattening if needed
        #zero out the gradients
        optimizer.zero_grad()

        #Calculate the output
        output = model(images)

        #Calculate the loss of the output, note: labels are the AC
        loss = criterion(output, labels)

        #Backpropagation (this is the part where we take the gradi
        loss.backward()

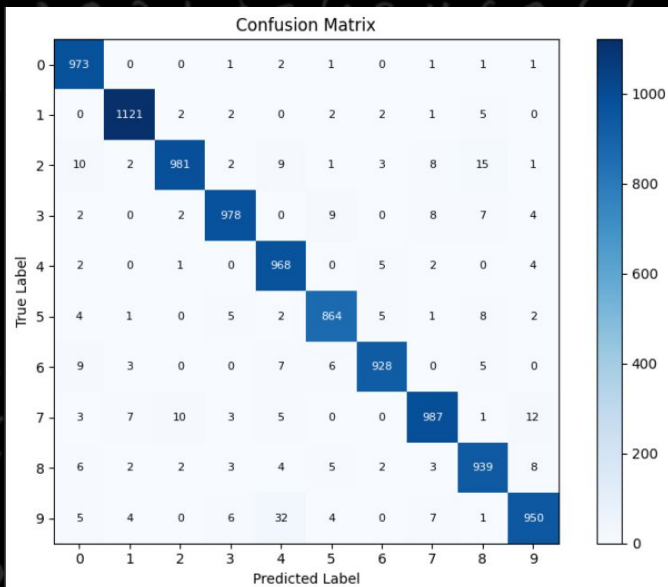
        running_loss.append(loss.item())
        epoch_loss += loss.item()

    #Let's update the weights here
    optimizer.step()
```

test

Test Accuracy: 96.89%

Confusion Matrix: tensor([[973, 0, 0, 1, 2, 1, 0, 1, 1, 1],
[0, 1121, 2, 2, 0, 2, 2, 1, 5, 0],
[10, 2, 981, 2, 9, 1, 3, 8, 15, 1],
[2, 0, 2, 978, 0, 9, 0, 8, 7, 4],
[2, 0, 1, 0, 968, 0, 5, 2, 0, 4],
[4, 1, 0, 5, 2, 864, 5, 1, 8, 2],
[9, 3, 0, 0, 7, 6, 928, 0, 5, 0],
[3, 7, 10, 3, 5, 0, 0, 987, 1, 12],
[6, 2, 2, 3, 4, 5, 2, 3, 939, 8],
[5, 4, 0, 6, 32, 4, 0, 7, 1, 950]])



```
model.eval()
correct = 0
total = 0
num_classes = 10
conf_matrix = torch.zeros(num_classes, num_classes, dtype=torch.int64)

#ensure gradients won't get changed
with torch.no_grad():
    for images, labels in testloader:
        images, labels = images.to(device), labels.to(device)

        #===== YOUR CODE HERE =====
        #do any preprocessing if necessary

        #calculate the output
        output = model(images)
        _, predicted = torch.max(output, dim=1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    for t, p in zip(labels.view(-1), predicted.view(-1)):
        conf_matrix[t.long(), p.long()] += 1
```

future?

grid search

there is apparently an emnist dataset that
includes letters too



THANKS